

NEXPIRE

AI Food Rescue Platform

An end-to-end technical, algorithmic, and architectural guide to the predictive food waste redistribution ecosystem.

0.5 kg

FOOD SAVED / RESCUED
UNIT

2.5 kg

CO₂E OFFSET / KG WASTE

180s

ATOMIC REDIS TTL LOCK

100%

SUBSIDIZED NGO BYPASS

PROJECT NAME

NEXPIRE (Formerly ResQ-Chain)

PRIMARY STACK

FastAPI + Postgres/PostGIS + Redis + Celery + Scikit-Learn

IMPLEMENTATION STATUS

All Phases Complete (Phase 0 – Phase 7)

DEPLOYMENT TOPOLOGY

Docker Compose (4 Services, Host Port 8001)

1. Executive Summary & Domain Knowledge

Domain Context

NEXPIRE is an intelligent, automated food waste reduction and redistribution platform designed to bridge the gap between commercial perishable inventory loss, community food rescue, and charitable hunger relief.

The Global Food Waste Paradox

Globally, **1.3 billion tons** of food is discarded annually (~33% of all food produced), generating 8–10% of global greenhouse emissions. Concurrently, hundreds of millions suffer from food insecurity. Retail stores discard perfectly nutritious items simply because they have reached a commercial "best before" cutoff or foot traffic slowed due to weather events.

The Four Critical Inefficiencies in Retail Food Disposal

1. The "Expiration Cliff" & Static Sticker Markdowns

Store workers typically walk aisles late in the afternoon slapping fixed 20% or 50% discount stickers onto products. This process is labor-intensive, unscientific, and usually happens too late to capture shoppers before closing.

2. Weather Sensitivity & Foot Traffic Shocks

Severe rain, heatwaves, or snowstorms drastically reduce store foot traffic. While demand drops by 40–70%, shelf inventory decay accelerates, causing mass disposal that static models cannot anticipate.

3. Concurrency Double-Claiming Failures

When high-value perishables (e.g., artisanal meat or bakery) are blasted out on discount apps, simultaneous claims cause severe stock conflicts and customer frustration if the backend lacks atomic locking.

4. Disconnected Shelter & NGO Redistribution

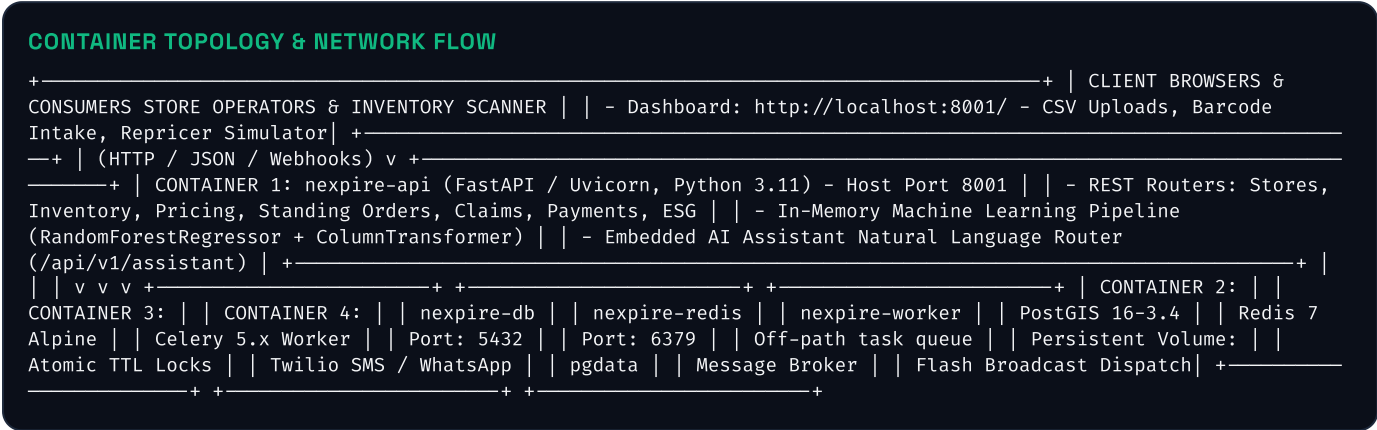
Charities, food banks, and shelters receive unpredictable donations of items they cannot use (e.g., 200kg of cabbage when they need protein). There is no automated subscription or priority reservation pipeline.

The NEXPIRE Automated Solution Cycle

- 1. Monitor:** Scans PostgreSQL inventory databases continuously for batches nearing perishable expiration.
- 2. Predict:** A Scikit-Learn Machine Learning pipeline calculates item risk scores by evaluating category shelf-life decay, stock depth, and live ambient temperature.
- 3. Price:** Computes the exact clearance discount required to maximize recovery while protecting retailer margins.
- 4. Alert:** Triggers hyper-local SMS and WhatsApp Business broadcasts via Twilio to verified consumers within a PostGIS-calculated radius.
- 5. Reserve:** Users tap a tokenized link or reply "YES" to an SMS. An atomic Redis TTL lock holds the item for 180 seconds to prevent double-claiming.
- 6. Pay & Fulfill:** Consumers pay via Razorpay or Stripe and select Self-Pickup (QR code) or Local Delivery. Registered NGOs skip payment via subsidized allocation.
- 7. Quantify:** Aggregates environmental impact (kg food rescued, kg CO₂e offset) and outputs shareable social impact receipts.

2. System Architecture & Topology

NEXPIRE operates as a decoupled microservices-ready architecture managed via Docker Compose. The topology is optimized for sub-millisecond concurrency locking, fast geospatial querying, and asynchronous notification broadcasting.



Core Architectural Components

SERVICE	CONTAINER NAME	PORT	KEY RESPONSIBILITIES
FastAPI Web Core	nexpire-api	8001:8000	High-throughput asynchronous ASGI web server, Pydantic v2 schemas, REST endpoints, static dashboard host.
PostgreSQL + PostGIS	nexpire-db	5432:5432	ACID relational database, spatial index engine (ST_DWithin), inventory batches, claims, standing orders.
Redis In-Memory Engine	nexpire-redis	6379:6379	High-concurrency atomic reservation lock (SET NX EX), Celery queue message broker, fast TTL manager.
Celery Task Worker	nexpire-worker	Internal	Background asynchronous queue worker executing batch flash broadcasts and bulk notification dispatches.

3. The AI Expiration Risk & Dynamic Pricing Engine

Standard markdown models treat 2 days to expiry identically across milk, bread, and canned goods. NEXPIRE introduces **Category-Relative Urgency Normalization**, translating raw expiry timelines into a universal urgency signal (\$0.0\$ to \$1.0\$).

3.1 The Category Urgency Normalization

Curve Type 1: Linear-Convex (Perishables)

Applies to: Dairy, Bakery, Meat & Seafood, Fruits & Veggies, Ready-to-Eat.

Urgency accelerates dramatically in the final 20% of shelf life, matching compounding bacterial growth and sensory decay.

$$\text{Urgency} = (1 - \text{DaysRemaining} / \text{ShelfLife})^{1.4}$$

Curve Type 2: Sigmoid Ramp (Pantry & Long-Shelf)

Applies to: Food Grains, Oil & Masala, Snacks, Beverages, Canned.

Urgency stays near 0% across 90% of product life, ramping up steeply only in the final 5–10% to prevent premature clearance.

$$\text{Urgency} = 1 / (1 + e^{-(k * (\text{FractionElapsed} - 0.92))})$$

3.2 Mathematical Formulation of Spoilage Risk

The total risk score is a composite function of decay, weather temperature, remaining volume, and physical grade:

$$F_{\text{decay}} = \exp(-0.25 * (\text{DaysToExpiry} / \text{CategoryPerishabilityIndex}))$$

$$F_{\text{temp}} = 1.0 + \max(0, (\text{AmbientTemp}_C - 25.0) * 0.02)$$

$$F_{\text{qty}} = 1.0 + \min(0.5, (\text{RemainingQuantity} / 100.0) * 0.2)$$

$$\text{Total Spoilage Risk} = \text{clip}(F_{\text{decay}} * F_{\text{temp}} * F_{\text{qty}} * \text{PhysicalConditionMultiplier} + N(0, 0.03), 0.05, 0.99)$$

3.3 Risk-to-Discount Mapping Tiers

RISK LEVEL	CALCULATED RISK (\$R\$)	DYNAMIC MARKDOWN %	CLEARANCE VELOCITY TARGET
LOW RISK	$R < 0.20$	0% - 10%	Normal store browsing clearance.
MEDIUM RISK	$0.20 \leq R < 0.45$	15% - 30%	In-store display highlight & mobile feed recommendation.
HIGH RISK	$0.45 \leq R < 0.75$	35% - 60%	Hyper-local SMS/WhatsApp flash broadcast dispatch.
CRITICAL RISK	$R \geq 0.75$	65% - 90%	Emergency flash sale / Standing order shelter match.

4. Phase-by-Phase Technical Implementation

Phases 0 - 7

Phase 0: Infrastructure & Container Orchestration

- **PostgreSQL 16 with PostGIS 3.4** configured with automated database seeding via `init.sql`.
- **Redis 7 Alpine** configured with health check ping intervals of 5s and volume persistence.
- **Proof of Life Check:** `GET /health` queries both engines and returns `{"database":"ok","redis":"ok","healthy":true}`.

Phase 1: Store & Inventory Core Engine

- CRUD operations for `Store` and `Batch` entities using SQLAlchemy ORM and Pydantic v2 schemas.
- Automatic `current_price` calculation based on `original_price` and `discount_percentage`.
- Streamed CSV ingestion (`POST /api/v1/inventory/upload-csv`) with row-level error reporting that preserves valid rows.

Phase 2: ML Expiration Risk & Dynamic Discounting

- Scikit-Learn Multi-Output `RandomForestRegressor` serialized via `joblib` at `app/ml/artifacts/pricing_model.joblib`.
- Live weather integration querying OpenWeatherMap API for ambient temperatures, with fallback handling.
- Interactive batch repricing endpoint: `POST /api/v1/ml/reprice-batch/{batch_id}`.

Phase 3: Omnichannel Notification Engine

- Integrated Twilio SMS and WhatsApp Business API supporting formatted messages with dynamic tokenized links.
- Inbound SMS webhook (`POST /api/v1/notifications/twilio-inbound`) allowing single-tap claiming by replying "YES".
- Celery background worker task (`dispatch_batch_notifications_task`) processing dispatches off the HTTP thread.

Phase 4: Standing Orders & NGO Priority Allocation

- Rule-based subscription engine for shelters and food banks (category filters, minimum quantities, priority windows).
- Subsidized matches flagged with `is_subsidized = True`, completely bypassing commercial payment gates.
- Automated matching evaluator: `POST /api/v1/standing-orders/evaluate`.

Phase 5: Concurrency-Safe Marketplace & Geo-Routing

- PostGIS `ST_DWithin` geospatial queries searching inventory batches within walking/driving radii.
- Atomic Redis reservation locking (`SET NX EX claim:lock:{batch_id}`) with configurable TTL (180s) to eliminate double-claims.
- Single-use URL-safe tokenized claim records (`Claim` model).

Phase 6: Multi-Rail Payment Gateway & Fulfillment

- Dual gateway support: **Razorpay** (INR/paise) and **Stripe** (USD/cents) with mock fallbacks for testing.
- HMAC SHA-256 webhook verification atomically updating claim status to `PAID`.
- Dual fulfillment routing: In-Store Self-Pickup (QR code generator) and Local Delivery routing.

Phase 7: Eco-Impact Analytics & Hardening

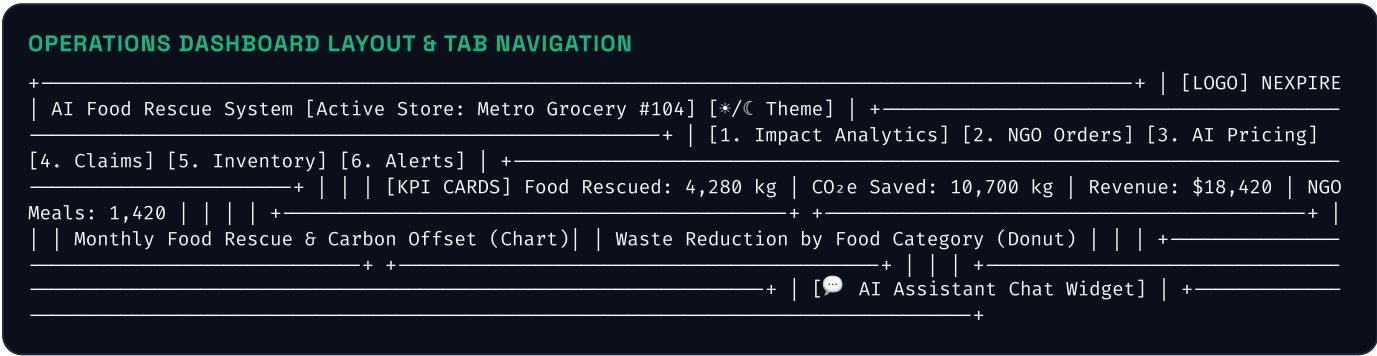
- ESG impact calculation: 0.5 kg waste prevented per item, 2.5 kg CO₂e offset per kg food rescued.
- Full platform metrics at `GET /api/v1/analytics/summary` and store-level breakdown at `/api/v1/analytics/store/{id}`.
- 36+ passing automated test suites covering all workflows and error conditions.

5. The User Interface (UI) & User Experience (UX) Deep-Dive

NEXPIRE provides two purpose-built web interfaces: the **Operations & Store Dashboard** (for retail managers, analysts, and operators) and the **Consumer Claim Portal** (a lightweight, mobile-optimized landing page for shoppers).

5.1 The Operations & Store Dashboard (/)

Engineered with vanilla CSS, glassmorphic panels, CSS variables, Outfit/Space Grotesk typography, and a theme switcher that stores state in `localStorage` and dynamically refreshes Chart.js colors.



Walkthrough of Dashboard Tabs

- **Tab 1: Impact & ESG Analytics (#impact-tab)**: Displays executive-ready sustainability metrics. Includes dual-axis historical trend charts (Food Rescued vs. CO₂e Averted) and category distribution donuts rendered using Chart.js. Offers instant ESG report generation.
- **Tab 2: Standing Orders & NGO Priority Rescue (#ngo-tab)**: Enables charitable organizations to register standing subscription filters. Operators can click "Evaluate Matches" to automatically pair expiring batches with shelter needs at 100% subsidy.
- **Tab 3: AI Dynamic Pricing Simulator (#ml-tab)**: Allows operators to simulate the Scikit-Learn pricing model interactively. Features sliders for remaining shelf hours, batch volume, and weather temperature. Includes a "Reprice Batch" button that writes new AI discounts to PostgreSQL.
- **Tab 4: Claims & Geo-Marketplace (#claims-tab)**: Visualizes customer reservations, real-time PostGIS radius filters, active Redis locks, and a multi-tier descending price Dutch auction simulator.
- **Tab 5: Store & Inventory Manager (#inventory-tab)**: Full CRUD inventory management. Includes color-coded urgency badges, a "New Batch" modal, CSV drag-and-drop batch upload, and a simulated shelf barcode scanner.
- **Tab 6: Flash Alerts & Notifications (#notify-tab)**: A command center for dispatching SMS and WhatsApp flash sales via Twilio, featuring an interactive webhook simulator to test two-way "Reply YES" claiming.
- **Floating Widget: NEXPIRE AI Assistant**: A persistent chat widget powered by `/api/v1/assistant` . Guides users through active deals, claiming steps, payment inquiries, and food safety FAQs.

5.2 The Consumer Claim Portal (`/claim/{token}`)

Consumer Flow

When a customer taps an SMS or WhatsApp notification link, they open a lightweight, mobile-first web page. The page requires **no app download and no account creation**, removing friction to ensure rapid claiming before product spoilage.

CONSUMER MOBILE CLAIM PAGE WIREFRAME

+ | NEXPIRE FLASH RESCUE | | | [⚡ FLASH RESCUE] Expires in: 02:45 | | |
| Artisan Sourdough Loaf (Pack of 2) | | Downtown Bakery Hub | | | Original: \$6.00 Discount: -60% | | NOW: \$2.40
| | | [=====] | | Reserved for you for 180 seconds | | | Fulfillment Preference: | | (o) Self-Pickup at
Store (Free) | | () Local Green Courier (+\$1.50) | | | [PAY \$2.40 & SECURE CLAIM] | | | Guaranteed Fresh •
Zero Food Waste | +-----+

Key Features of the Consumer Claim Flow

- 1. Live Redis TTL Reservation Timer:** A prominent visual progress bar and countdown timer shows the user's 3-minute lock window. If checkout is abandoned, the lock auto-releases.
- 2. Multi-Tier Descending Price Ticker:** For high-volume auction items, the price visibly decreases at set intervals, incentivizing immediate conversion.
- 3. Integrated Multi-Rail Payment:** Clicking the pay button opens a secure payment modal (Razorpay in India, Stripe globally).
- 4. Subsidized NGO Bypass:** If opened via a shelter match token, the payment button is replaced with a prominent green *"Confirm Subsidized Pickup (Free)"* button.
- 5. Dual Fulfillment Confirmation:**
 - **Self-Pickup:** Generates a high-contrast QR code for store staff verification and provides one-tap navigation to the store.
 - **Local Delivery:** Prompts for delivery address, adds a delivery fee, and tracks courier status.
- 6. Shareable Social Impact Receipt:** Upon completion, generates a downloadable badge: *"🌱 I just rescued 1.5 kg of food and saved 3.7 kg CO₂e with NEXPIRE!"*

6. Database Schema & Data Models

The persistence layer is modeled in SQLAlchemy with PostgreSQL/PostGIS. Below is the relational entity specification:

TABLE NAME	FIELD NAME	DATA TYPE	DESCRIPTION & CONSTRAINTS
stores	id	Integer (PK)	Unique store identifier.
	name / address	String	Store brand name and street address.
	latitude / longitude	Float	Spatial coordinates used by PostGIS for radius routing.
	contact_phone	String	Store manager phone number.
batches	id / store_id	Integer (PK / FK)	Batch ID and foreign key reference to stores.id .
	sku / item_name	String	Product stock code and human-readable name.
	category	String	Category: dairy , bakery , produce , meat , etc.
	remaining_quantity	Integer	Remaining stock count in batch.
	original_price / current_price	Float	Original retail price and current AI-discounted price.
	expiration_date / status	DateTime / String	Expiry timestamp and state: ACTIVE , DISCOUNTED , CLAIMED .
claims	id / batch_id	Integer (PK / FK)	Claim ID and foreign key reference to batches.id .
	claim_token	String (Unique)	URL-safe random token used in single-tap links.
	status	String	Status: RESERVED , PENDING_PAYMENT , PAID , FULFILLED .
	fulfillment_type	String	Choice: PICKUP or DELIVERY .
	reserved_until	DateTime	TTL mirror timestamp matching the Redis key expiration.
	is_subsidized	Boolean	True if claim is an NGO allocation skipping payment gates.
standing_orders	id / organization_name	Integer (PK) / String	Charity ID and shelter/food bank legal name.
	category_filter	String	Preferred food categories (e.g. produce , bakery).
	priority_window_hours	Integer	Hours before expiration to allocate priority reservation.

7. REST API Endpoints Reference

API V1

All endpoints are prefixed with `/api/v1` . Interactive OpenAPI documentation is hosted at `/docs` .

METHOD	PATH	DESCRIPTION
GET	<code>/health</code>	Proof-of-life check verifying Postgres and Redis connectivity.
POST	<code>/api/v1/stores</code>	Register a new store location with geospatial coordinates.
GET	<code>/api/v1/inventory/batches</code>	Query batches filtered by store, category, and expiration status.
POST	<code>/api/v1/inventory/upload-csv</code>	Streamed multipart CSV ingestion with row-level diagnostics.
POST	<code>/api/v1/ml/predict-discount</code>	Calculate risk score (\$0.0-1.0\$) and recommended discount %.
POST	<code>/api/v1/ml/reprice-batch/{id}</code>	Fetch live weather, calculate AI discount, and update Postgres batch.
POST	<code>/api/v1/notifications/broadcast/{id}</code>	Queue Celery background SMS/WhatsApp broadcast to nearby buyers.
POST	<code>/api/v1/notifications/twilio-inbound</code>	Twilio webhook processing single-tap "reply YES" SMS claiming.
POST	<code>/api/v1/claims/geo-search</code>	PostGIS spatial search returning deals within given kilometer radius.
POST	<code>/api/v1/claims</code>	Create tokenized claim reservation with atomic Redis TTL lock.
GET	<code>/api/v1/claims/{token}</code>	Resolve public consumer claim details by token.
POST	<code>/api/v1/payments/checkout-session</code>	Create Razorpay Order or Stripe Session (supports NGO bypass).
GET	<code>/api/v1/analytics/summary</code>	Platform-wide food waste prevented, CO ₂ e saved, and NGO meals.
POST	<code>/api/v1/assistant/chat</code>	Conversational query engine for deals, claims, and food safety FAQs.

8. Deployment, Operational Runbook & Verification

8.1 Production Deployment with Docker Compose

```
# 1. Clone workspace and setup environment variables
cp .env.example .env

# 2. Build and start all 4 containers in detached mode
docker compose up --build -d

# 3. Verify health status of all nodes
curl http://localhost:8001/health
# Response: {"database":"ok","redis":"ok","healthy":true}

# 4. View real-time container status
docker compose ps
```

8.2 Running the Automated Test Suite

The codebase includes 36 comprehensive test suites validating all API phases, ML models, and edge cases:

```
# Execute pytest inside the running API container
docker compose exec api pytest -v --cov=app

# Run focused tests on ML pricing engine
docker compose exec api pytest tests/test_pricing.py -v
```

8.3 Primary Service URLs Reference

SERVICE NAME	URL	DESCRIPTION
Operations Dashboard	http://localhost:8001/	Store inventory, ML simulator, claims radar, NGO orders, analytics.
Interactive Swagger UI	http://localhost:8001/docs	FastAPI interactive endpoint testing and OpenAPI schema viewer.
Alternative ReDoc Specs	http://localhost:8001/redoc	Formatted OpenAPI documentation for developer onboarding.
API Health Check	http://localhost:8001/health	Kubernetes/Docker liveness probe verifying DB and Redis nodes.
Consumer Claim Demo	http://localhost:8001/claim/new	Live demo of consumer claim page with QR and payment checkout.

Summary & Conclusion

NEXPIRE bridges enterprise retail inventory systems, predictive machine learning, high-concurrency reservation locking, and community hunger relief into a single automated ecosystem. It delivers tangible economic recovery to retailers while preventing thousands of tons of edible food from entering landfills.